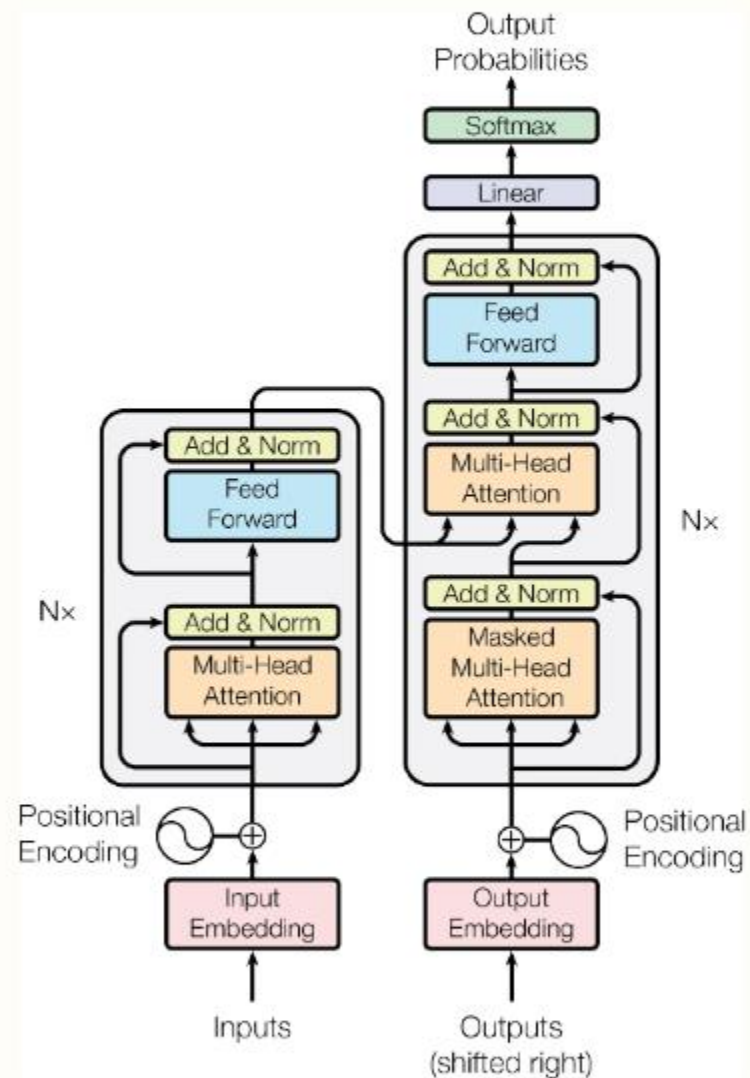


The Original Transformer Architecture

The 2017 paper assembled familiar components into an encoder-decoder system.

READ THE PUBLISHED DIAGRAM FROM BOTTOM TO TOP



What to recognize

- 1 Encode the source sequence**
The left stack alternates self-attention and a position-wise feedforward network.
- 2 Mask the target-side self-attention**
The decoder can use earlier outputs, but it cannot look at future target tokens.
- 3 Revisit the encoder through cross-attention**
The middle decoder attention block queries the source representation at every output step.
- 4 Produce next-token probabilities**
A linear vocabulary head and softmax turn the final decoder state into a distribution.

Decoder-only LLMs keep the masked decoder stack—but remove the encoder and cross-attention.

Transformer Concepts

The diagram connects representations, attention, layer operations, and prediction.

IDEAS USED IN THE ORIGINAL ENCODER-DECODER ARCHITECTURE

Input representations

Token embeddings and positional information form the input vectors.

Attention calculation

Learned projections produce queries, keys, and values.
Softmax weights from scaled query-key scores combine the values.

Attention roles

Encoder self-attention, decoder self-attention with a causal mask, and cross-attention use different available information.

Transformer layers

Multiple heads, feed-forward networks, residual connections, and normalization build the encoder and decoder layers.

Prediction and generation

Shifted target inputs support parallel training. A vocabulary projection and softmax produce next-token probabilities.

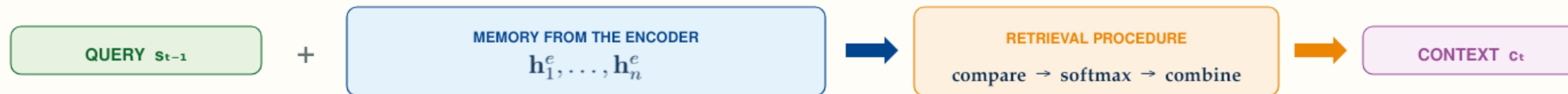
The following slides develop these ideas, beginning with attention.

Cross-Attention and Self-Attention

The retrieval procedure stays the same. The source of the query and memory changes.

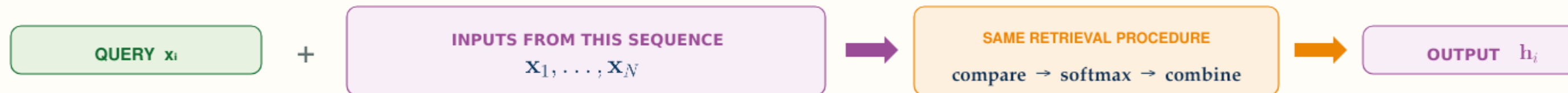
ATTENTION INPUTS

CROSS-ATTENTION



A decoder state retrieves source information from a different sequence.

SELF-ATTENTION



Each position retrieves information from allowed positions in the same sequence.

Causal attention sees the current and earlier positions.
A bidirectional encoder may attend to positions on both sides.

The Transformer makes the comparison and retrieval roles explicit through queries, keys, and values.

Database Lookup and Attention

A query identifies relevant entries. Keys support matching, and values contain the information to retrieve.

DATABASE LOOKUP

QUERY

book_id = 'BK205'

Find the title and shelf
for this book ID.



BOOK CATALOGUE

book_id (key)	Title (value)	Shelf (value)
BK101	Linear Algebra	A2
BK205	Language Models	B4
BK310	Optimization	C1



RETURNED RECORD

Language Models
Shelf B4

SOFT ATTENTION

QUERY VECTOR

q_i

Compare with each key.
Softmax gives weights.



Key vector	Value vector	Weight
k_1	v_1	0.10
k_2	v_2	0.80
k_3	v_3	0.10



WEIGHTED RESULT

$$0.10v_1 + 0.80v_2 + 0.10v_3$$

Values contribute in
proportion to their
weights.

Illustrative weights. Attention combines vector values, not catalogue text.

Soft retrieval combines several values. Differentiability lets training adjust the matching rule.

Role-Specific Representations

SGNS and attention use separate vectors for the two sides of a dot-product score.

RELATION TO SGNS

SGNS: STORED EMBEDDINGS

Word	Target vector	Context vector
doctor	$\mathbf{W}_{\text{doctor}}$	$\mathbf{C}_{\text{doctor}}$
medicine	$\mathbf{W}_{\text{medicine}}$	$\mathbf{C}_{\text{medicine}}$

The same word has a separate learned vector in each embedding table.

DOT-PRODUCT SCORE

$$\mathbf{W}_{\text{doctor}} \cdot \mathbf{C}_{\text{medicine}}$$

Target “doctor” with context “medicine”

SGNS learns word-type embeddings. Attention computes vectors from the current token representations.

SELF-ATTENTION: COMPUTED VECTORS

INPUT REPRESENTATION AT POSITION i
 $\mathbf{x}_i$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}_Q$$

$$\mathbf{k}_i = \mathbf{x}_i \mathbf{W}_K$$

Different learned matrices allow different vectors for the same token.

DOT-PRODUCT SCORE

$$\mathbf{q}_i \cdot \mathbf{k}_j$$

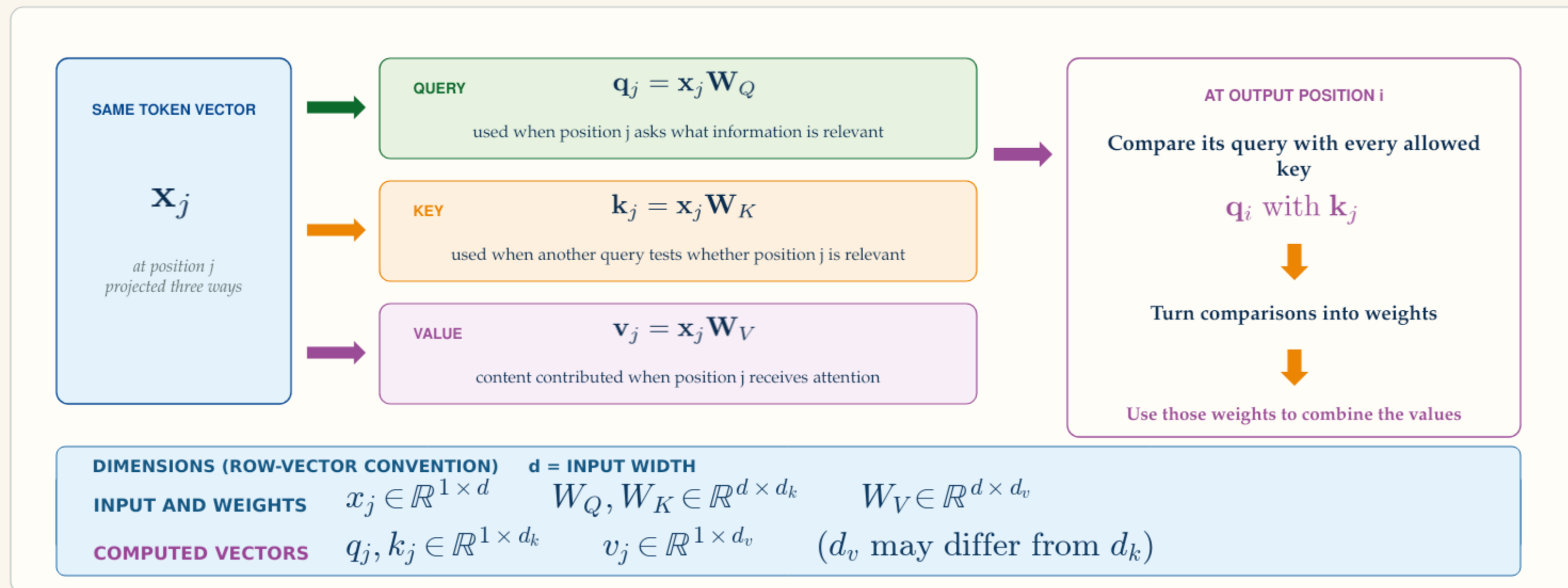
Query at position i with key at position j

Different roles motivate separate query and key projections. Values carry the content to retrieve.

Query, Key, and Value Projections

Each token representation is projected into three roles used by the attention procedure.

PROJECTION ROLES



We will apply these projections to the target sequence of a translation decoder.

Vaswani et al. (2017)

Hindi-to-English Translation Example

The encoder holds the Hindi source while the decoder generates the English target.

A SOURCE SENTENCE AND ONE TARGET STEP

HINDI SOURCE • ENCODER

बिल्ली सोई।

Available at every decoder step



ENGLISH TARGET • DECODER

The cat slept.

Generated one token at a time

DECODER STEP: PREDICT “slept”

<BOS>

The

cat



slept

DECODER SELF-ATTENTION

English prefix: <BOS>, The, cat

DECODER CROSS-ATTENTION

Complete Hindi source

We first examine “The cat slept” as a complete encoder input.

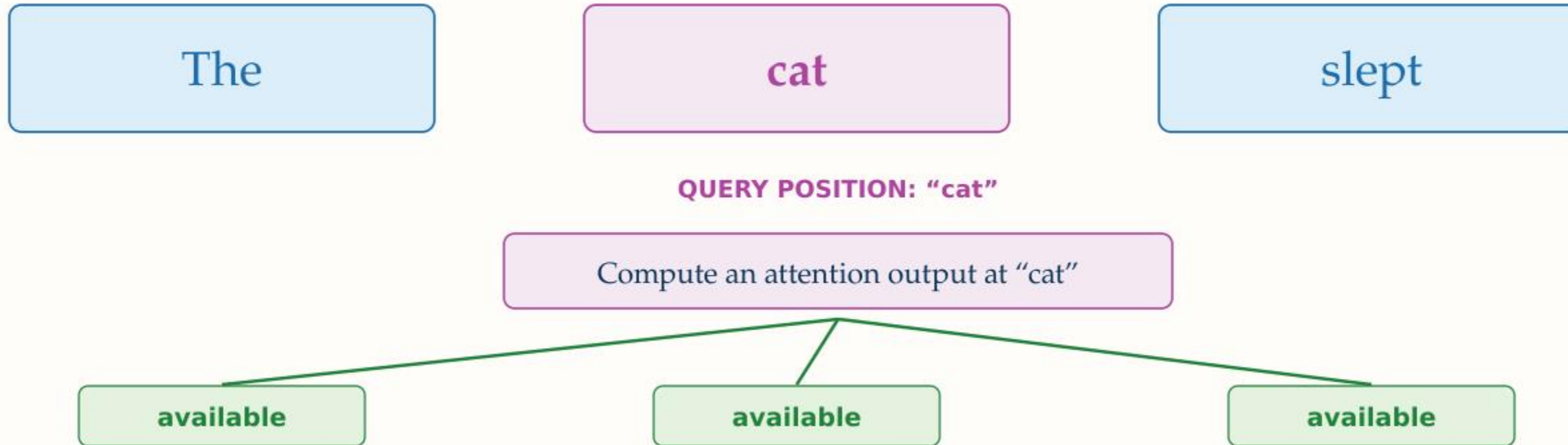
Encoder Self-Attention

An encoder position may use the complete input sentence to form a contextual representation.

THE SAME WORDS AS AN ENCODER INPUT

ENCODER INPUT

For this example, the English sentence is the complete encoder input.



All three input positions can contribute to this encoder attention output.

The query at "cat" may include "slept" because "slept" is already in the encoder input.

Input Representations

The example fixes small input vectors and learned projection matrices for one attention head.

THREE TOKEN ROWS AND THREE PROJECTION ROLES

TOY INPUT ROWS

Each token has an input row of width $d = 3$.

Position	Input row
The	$[1 \ 0 \ 0]$
cat	$[1 \ 1 \ 0]$
slept	$[1 \ 1 \ 1]$

X has shape 3×3 .

Chosen numbers isolate the arithmetic. Positional information is omitted here.

SHARED PROJECTION MATRICES

Each matrix is applied to every token row.

$$W_Q = \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{bmatrix}$$

3×2

$$W_K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$$

3×2

$$W_V = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}$$

3×2

Queries and keys have width 2; values also have width 2 here.

The projection matrices are parameters; queries, keys, and values are computed from X .

The Query at “cat”

The query vector specifies the attention computation for the position being updated.

ONE QUERY, THEN THREE COMPARISONS

QUERY POSITION

Input row for “cat”

$$x_2 = \begin{bmatrix} 1 & 1 & 0 \end{bmatrix}$$

Apply the learned query projection.

$$\begin{aligned} q_2 &= x_2 W_Q \\ &= \begin{bmatrix} 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 \end{bmatrix} \end{aligned}$$

The query is computed for this input; the projection matrix is shared and learned.

NEXT: COMPARE THIS QUERY WITH THREE KEYS

key for “The”

key for “cat”

key for “slept”

The query at “cat” will be compared with the keys for “The,” “cat,” and “slept.”

Encoder Keys and Scores

The query at “cat” is compared with the key computed at every input position.

THREE KEY VECTORS, THREE DOT PRODUCTS

QUERY AT “cat”

$$q_2 = \begin{bmatrix} 1 & 1 \end{bmatrix}$$

SHARED KEY MATRIX

$$W_K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$$

COMPUTE EACH KEY

$$k_j = x_j W_K$$

POSITION 1 • The

$$x_1 = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$$

$$k_1 = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

$$q_2 k_1^T = 1$$

POSITION 2 • cat

$$x_2 = \begin{bmatrix} 1 & 1 & 0 \end{bmatrix}$$

$$k_2 = \begin{bmatrix} 1 & 1 \end{bmatrix}$$

$$q_2 k_2^T = 2$$

POSITION 3 • slept

$$x_3 = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$$

$$k_3 = \begin{bmatrix} 1 & 1 \end{bmatrix}$$

$$q_2 k_3^T = 2$$

In this toy projection, the keys for “cat” and “slept” happen to be equal.

The query–key comparisons give unscaled scores 1, 2, and 2.

Encoder Values

The value vectors provide the content that attention will combine after scoring.

THREE VALUE VECTORS FROM THE SAME INPUT ROWS

VALUE PROJECTION

Each input row also produces a value using the same projection matrix.

$$v_j = x_j W_V$$

POSITION 1 • The

$$x_1 = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$$

$$v_1 = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

POSITION 2 • cat

$$x_2 = \begin{bmatrix} 1 & 1 & 0 \end{bmatrix}$$

$$v_2 = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

POSITION 3 • slept

$$x_3 = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$$

$$v_3 = \begin{bmatrix} 1 & 1 \end{bmatrix}$$

The keys for “cat” and “slept” match in this toy example.
Their values differ: the second coordinate appears only in the value for “slept.”

Keys determine the weights. Those weights will combine the values.

Encoder Attention Weights

Scale the three dot products and normalize them across the complete input sentence.

SCORES AND WEIGHTS AT POSITION 2

SCALED SCORES FOR THE QUERY AT "cat"

$$e_{2j} = q_2 k_j^T / \sqrt{2}$$

All three encoder positions are available, so softmax uses all three scores.

$$d_k = 2$$

	The	cat	slept
Dot product	1	2	2
Scaled score	0.707	1.414	1.414
Softmax weight	0.198	0.401	0.401

$$\alpha_{2,:} = \text{softmax}(e_{2,:}) \approx [0.198 \quad 0.401 \quad 0.401]$$

The three weights sum to one and will be applied to the three value vectors.

Encoder Context Vector

The attention weights combine the three value vectors for the query at “cat.”

WEIGHTED VALUE CONTRIBUTIONS

WEIGHTED SUM FOR THE QUERY AT “cat”

$$h_2 = \alpha_{21}v_1 + \alpha_{22}v_2 + \alpha_{23}v_3$$

Use the weights and values just computed.

The

WEIGHT × VALUE

$$0.198 \begin{bmatrix} 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0.198 & 0 \end{bmatrix}$$

cat

WEIGHT × VALUE

$$0.401 \begin{bmatrix} 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0.401 & 0 \end{bmatrix}$$

slept

WEIGHT × VALUE

$$0.401 \begin{bmatrix} 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 0.401 & 0.401 \end{bmatrix}$$

ONE HEAD’S OUTPUT AT “cat”

$$h_2 \approx \begin{bmatrix} 1.000 & 0.401 \end{bmatrix}$$

The second coordinate comes from the available input “slept.”

This is one head’s context vector. Later block operations produce the model-width representation.

Parallel Encoder Self-Attention

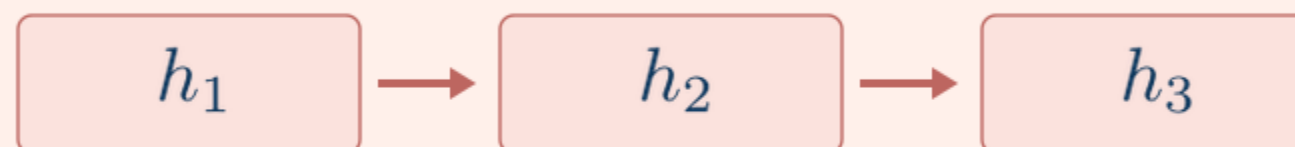
The encoder can calculate attention outputs for all input positions together.

RECURRENCE AND MATRIX COMPUTATION

DEPENDENCIES WITHIN ONE LAYER

RECURRENT ENCODER

Each state depends on the preceding state.



ENCODER SELF-ATTENTION

All three input rows are present.

PROJECT ALL ROWS

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

3×2 for Q, K, and V

SCORE ALL QUERIES

$$S = QK^T / \sqrt{2}$$

3×3 score matrix

COMBINE ALL VALUES

$$A = \text{softmax}_{\text{row}}(S)$$

$$H = AV \quad 3 \times 2 \text{ output}$$

Matrix products and independent softmax rows can be evaluated in parallel on suitable hardware.

Attention has no state-to-state recurrence across positions within this layer.

Shifted Decoder Targets

Each decoder position receives an input token and predicts the following target token.

INPUTS, LABELS, AND AVAILABLE PREFIXES

TEACHER-FORCED TRAINING INPUT

The English target is shifted by one position to supply the decoder input.

Position	Input token	Next-token label	Available target-side prefix
1	⟨BOS⟩	The	⟨BOS⟩
2	The	cat	⟨BOS⟩, The
3	cat	slept	⟨BOS⟩, The, cat
4	slept	.	⟨BOS⟩, The, cat, slept

At position 3, “cat” is an input and “slept” is the label to predict.

Training supplies all input positions together, while each query uses only its permitted prefix.

Attention in Translation

Each attention operation has a defined source for its queries, keys, and values.

SOURCES AND ACCESSIBLE POSITIONS

In this translation task, Hindi is the encoder input and English is the decoder target.

The preceding English encoder calculation was a separate illustrative task.

Attention operation	Query source	Key and value source	Available positions
Encoder self-attention	Hindi source	Hindi source	Complete Hindi input
Decoder self-attention	English target prefix	English target prefix	Current and earlier target inputs
Cross-attention	Decoder representation	Hindi encoder outputs	Complete Hindi source

At “cat,” decoder self-attention uses the English prefix; cross-attention may read the full Hindi source.

The decoder combines permitted target context with information from the encoder.

Decoder Training Constraints

The decoder receives shifted target inputs together, but each prediction has limited access.

PARALLEL COMPUTATION WITH CAUSAL ACCESS

TWO REQUIREMENTS DURING TEACHER-FORCED TRAINING

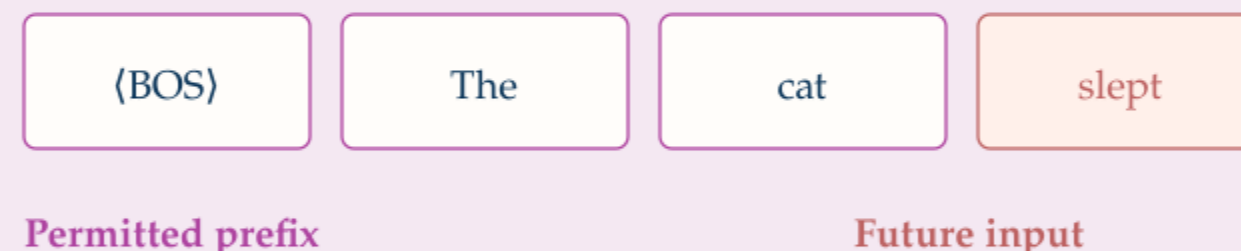
MULTIPLE PREDICTIONS

The target sentence supplies a next-token label at each position.

<BOS>	predicts	The
The	predicts	cat
cat	predicts	slept

CAUSAL ACCESS

At “cat,” the future input “slept” must not be visible to the query.



Can all decoder query rows be computed together while restricting each query to its available prefix?

Each decoder query must use only its available shifted input prefix.

Toy Decoder Representations

Reuse the toy encoder numbers as given and add a zero ⟨BOS⟩ input row.

INPUTS AND SHARED PROJECTIONS

SHIFTED DECODER INPUT: ⟨BOS⟩, THE, CAT, SLEPT

GIVEN INPUT, KEY, AND VALUE ROWS

Token	Input row	Key	Value
⟨BOS⟩	$\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 \end{bmatrix}$
The	$\begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \end{bmatrix}$
cat	$\begin{bmatrix} 1 & 1 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \end{bmatrix}$
slept	$\begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 1 \end{bmatrix}$

X is 4×3 ; K and V are 4×2 .

SHARED PROJECTION MATRICES

W_Q $\begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{bmatrix}$ 3×2	W_K $\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$ 3×2	W_V $\begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}$ 3×2
--	--	--

At “cat,” the query is:

$$q_3 = x_3 W_Q = \begin{bmatrix} 1 & 1 \end{bmatrix}$$

Only the future input “slept” has a nonzero second value coordinate.

These numerical choices are reused for teaching; actual encoder and decoder parameters are separate.

Unmasked Attention Weights

Without a causal restriction, the decoder query at “cat” assigns weight to the future input.

SCALED SCORES AND SOFTMAX

QUERY AT “cat”

Use the query and keys already calculated.

$$q_3 = \begin{bmatrix} 1 & 1 \end{bmatrix}, \quad e_{3j} = q_3 k_j^T / \sqrt{2}$$

KEY PROJECTION REFERENCE $W_K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$

Key position	Key vector	Scaled score	Exponentiated score	Attention weight
⟨BOS⟩	$\begin{bmatrix} 0 & 0 \end{bmatrix}$	0	1.000	0.089
The	$\begin{bmatrix} 1 & 0 \end{bmatrix}$	0.707	2.028	0.180
cat	$\begin{bmatrix} 1 & 1 \end{bmatrix}$	1.414	4.113	0.365
slept	$\begin{bmatrix} 1 & 1 \end{bmatrix}$	1.414	4.113	0.365

The scores for The, cat, and slept reuse the encoder example; ⟨BOS⟩ adds a zero score.

The future “slept” input receives about 0.365 of the attention weight.

Unmasked Attention Output

The weighted value mixture carries information from an unavailable target input.

VALUE CONTRIBUTIONS

WEIGHTED VALUES FOR THE QUERY AT “cat”

Use the unmasked weights from the preceding slide and the given value vectors.

Input position	Weight	Value vector	Weighted contribution
⟨BOS⟩	0.089	$\begin{bmatrix} 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 \end{bmatrix}$
The	0.180	$\begin{bmatrix} 1 & 0 \end{bmatrix}$	$\begin{bmatrix} 0.180 & 0 \end{bmatrix}$
cat	0.365	$\begin{bmatrix} 1 & 0 \end{bmatrix}$	$\begin{bmatrix} 0.365 & 0 \end{bmatrix}$
slept	0.365	$\begin{bmatrix} 1 & 1 \end{bmatrix}$	$\begin{bmatrix} 0.365 & 0.365 \end{bmatrix}$

The second coordinate comes entirely from the future input “slept.”

UNMASKED OUTPUT

$$h_3 \approx \begin{bmatrix} 0.911 & 0.365 \end{bmatrix}$$

The sum uses unrounded weights; the displayed values are rounded.

Causal Attention Mask

Each decoder query may read its current and earlier shifted target inputs.

PERMITTED KEYS BY QUERY POSITION

ADDITIVE MASK FOR FOUR SHIFTED TARGET INPUTS

Rows are queries; columns are keys.

Query / Key	⟨BOS⟩	The	cat	slept
⟨BOS⟩	0	$-\infty$	$-\infty$	$-\infty$
The	0	0	$-\infty$	$-\infty$
cat	0	0	0	$-\infty$
slept	0	0	0	0

A zero retains the score; $-\infty$ excludes it before the row-wise softmax.

The same matrix calculation evaluates all query rows while excluding future positions.

Masked Attention Weights

Masking removes the future score before the softmax is evaluated.

RENORMALIZATION OVER THE PERMITTED PREFIX

QUERY AT "cat"

SCALED SCORES FROM THE UNMASKED CALCULATION

$$e_3 = [0 \quad 0.707 \quad 1.414 \quad 1.414]$$

CAUSAL MASK ROW

$$m_3 = [0 \quad 0 \quad 0 \quad -\infty]$$

Key position	Scaled score	Mask entry	Masked score	Exponentiated score	New weight
<BOS>	0	0	0	1.000	0.140
The	0.707	0	0.707	2.028	0.284
cat	1.414	0	1.414	4.113	0.576
slept	1.414	$-\infty$	$-\infty$	0	0

The permitted exponentials sum to $1 + 2.028 + 4.113 \approx 7.141$.

The future weight is zero; the three permitted weights sum to one.

Masked Attention Output

The permitted value vectors form a context vector without future target information.

VALUE CONTRIBUTIONS AND COMPARISON

WEIGHTED VALUES FOR THE QUERY AT “cat”

Use the masked weights and the same value vectors.

Input position	Masked weight	Value vector	Weighted contribution
⟨BOS⟩	0.140	$\begin{bmatrix} 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 \end{bmatrix}$
The	0.284	$\begin{bmatrix} 1 & 0 \end{bmatrix}$	$\begin{bmatrix} 0.284 & 0 \end{bmatrix}$
cat	0.576	$\begin{bmatrix} 1 & 0 \end{bmatrix}$	$\begin{bmatrix} 0.576 & 0 \end{bmatrix}$
slept	0	$\begin{bmatrix} 1 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 \end{bmatrix}$

The future “slept” input contributes the zero vector.

UNMASKED

$$h_3 \approx \begin{bmatrix} 0.911 & 0.365 \end{bmatrix}$$

MASKED

$$h_3 \approx \begin{bmatrix} 0.860 & 0 \end{bmatrix}$$

The second coordinate is zero, and the first changes because weights renormalize.

Masking and Normalization

Setting a future weight to zero after softmax does not reproduce masked attention by itself.

COMPARISON OF THREE PROCEDURES

QUERY AT “cat”: WEIGHT ON “slept” IS FORBIDDEN

Compare the distributions obtained by changing the score or by changing the final weight.

Procedure	Attention weights	Weight sum	Head output
No mask	$[0.089 \quad 0.180 \quad 0.365 \quad 0.365]$	1.000	$[0.911 \quad 0.365]$
Zero the last weight	$[0.089 \quad 0.180 \quad 0.365 \quad 0]$	0.635	$[0.546 \quad 0]$
Mask before softmax	$[0.140 \quad 0.284 \quad 0.576 \quad 0]$	1.000	$[0.860 \quad 0]$

Zeroing alone leaves weights that do not sum to one.

Their original denominator still depends on the future key.

Zeroing and renormalizing is equivalent in exact arithmetic to masking before softmax.

Applying the mask to scores normalizes directly over permitted positions.

Attention Weights Across Positions

During training, every decoder query has its own permitted prefix.

ATTENTION WEIGHT MATRIX

ROW-WISE SOFTMAX OF THE MASKED SCORE MATRIX

$$A = \text{softmax}_{\text{row}}(S)$$

Each row corresponds to one shifted decoder input.

Query	⟨BOS⟩	The	cat	slept	Next-token label
⟨BOS⟩	1.000	0	0	0	The
The	0.500	0.500	0	0	cat
cat	0.140	0.284	0.576	0	slept
slept	0.089	0.180	0.365	0.365	.

The “cat” row repeats the masked weights already computed.

The “slept” query may use that input because its next-token label is the period.

All rows sum to one before rounding, and future positions have exactly zero weight.

Parallel Decoder Computation

Teacher forcing supplies the shifted inputs; the causal mask restricts every query row.

MATRIX OPERATIONS WITHIN ONE ATTENTION HEAD

SHARED PROJECTIONS

$$Q = XW_Q, K = XW_K, V = XW_V$$

$X: 4 \times 3$; each $W: 3 \times 2$; $Q, K, V: 4 \times 2$

MASKED SCORES

$$S = QK^T / \sqrt{2} + M$$

S and $M: 4 \times 4$

ATTENTION OUTPUTS

$$A = \text{softmax}_{\text{row}}(S)$$

$$H = AV$$

$A: 4 \times 4$; $H: 4 \times 2$

OUTPUT ROWS FROM THE SAME TOY EXAMPLE

Query input	Attention output row	Next-token label
$\langle \text{BOS} \rangle$	$\begin{bmatrix} 0 & 0 \end{bmatrix}$	The
The	$\begin{bmatrix} 0.500 & 0 \end{bmatrix}$	cat
cat	$\begin{bmatrix} 0.860 & 0 \end{bmatrix}$	slept
slept	$\begin{bmatrix} 0.911 & 0.365 \end{bmatrix}$	.

These are head outputs; rows are evaluated together while each uses only its prefix.

Decoder Training and Inference

Training evaluates shifted target positions together; inference generates new target tokens sequentially.

TARGET-SIDE INFORMATION ACCESS

SAME PREDICTION AT THE INPUT "cat"

TEACHER-FORCED TRAINING

The full shifted English input is supplied:

⟨BOS⟩, The, cat, slept

All query positions are evaluated together within a layer.

At "cat," the target is "slept."

The mask excludes the future input "slept."

AUTOREGRESSIVE INFERENCE

The generated prefix is extended one token at a time.

⟨BOS⟩ → The

⟨BOS⟩, The → cat

⟨BOS⟩, The, cat → slept

At the last step, no future "slept" input exists.

The future reference token is supplied as input during teacher-forced training.

Causal access still applies when inference processes several prefix positions together.

The complete Hindi encoder output remains available to cross-attention in both settings.